

LAPORAN PRAKTIKUM STRUKTUR DATA

Single LinkedList



Oleh :

MUHAMMAD ARIF

NIM 2511532017

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 05 Mei 2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan praktikum “**Single LinkedList**” ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai dokumentasi dan pemahaman hasil kegiatan praktikum yang bertujuan untuk memahami bagaimana konsep dari Single LinkedList. Penulis juga terbuka untuk menerima saran dan kritik guna perbaikan di masa mendatang.

Penulis mengucapkan terima kasih sebesar-besarnya kepada:

1. DR. Wahyudi, S.T, M.T selaku pembimbing mata kuliah struktur data, yang telah memberikan bimbingan, ilmu, dan motivasi selama proses pembelajaran.

Padang, 05 Mei 2026

Muhammad Arif

DAFTAR PUSTAKA

LAPORAN PRAKTIKUM STRUKTUR DATA	i
KATA PENGANTAR.....	ii
DAFTAR PUSTAKA.....	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Landasan teori	3
2.2 Langkah Pengerjaan	3
BAB III KESIMPULAN.....	16
DAFTAR PUSTAKA.....	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengelolaan data yang efisien dan terstruktur merupakan fondasi utama dalam pengembangan perangkat lunak modern. Seiring meningkatnya kompleksitas sistem, pengembang tidak hanya dituntut memahami cara menyimpan data, tetapi juga mampu memisahkan logika penggunaan dari detail implementasi. Konsep *Abstract Data Type* (ADT) hadir sebagai solusi melalui prinsip abstraksi yang mendefinisikan tipe data berdasarkan perilaku dan operasi yang diizinkan, bukan representasi memorinya. Dengan memisahkan antarmuka (*interface*) dari implementasi, ADT mendukung *information hiding*, enkapsulasi, serta membentuk sistem yang modular, *reusable*, dan mudah diuji.

Meskipun bersifat teoretis, pemahaman ADT memerlukan implementasi langsung untuk menguji kebenaran operasi, menangani kondisi batas (*boundary conditions*), dan memvalidasi kontrak spesifikasi yang ditetapkan. Penguasaan konsep ini juga menjadi prasyarat penting sebelum mempelajari algoritma lanjutan maupun arsitektur perangkat lunak berbasis komponen. Oleh karena itu, praktikum ini dilaksanakan untuk memberikan pengalaman langsung dalam merancang, mengimplementasikan, dan menguji ADT sesuai spesifikasi teknis.

1.2 Tujuan

Tujuan dari dilakukannya praktikum ini adalah

1. Memahami struktur ADT Single LinkedList
2. Memahami langkah-langkah menulis ADT Single LinkedList

1.3 Manfaat

Manfaat yang diperoleh dari praktikum ini adalah

1. Meningkatkan pemahaman tentang ADT Single LinkedList
2. Meningkatkan pemahaman bagaimana cara menulis ADT Single LinkedList

BAB II

PEMBAHASAN

2.1 Landasan teori

Struktur data adalah cara penyimpanan, pengorganisasian, dan pengaturan data di dalam media penyimpanan komputer sehingga data tersebut dapat digunakan secara efisien. Sedangkan data adalah representasi dari fakta dunia nyata. Fakta atau keterangan tentang kenyataan yang disimpan, direkam atau direpresentasikan dalam bentuk tulisan, suara, gambar, sinyal atau simbol.

Definisi ADT (Abstract Data Type) sendiri dalam struktur data merupakan sebuah spesifikasi atau kontrak yang mendefinisikan antarmuka suatu tipe data. ADT tidak menentukan bagaimana tipe data diimplementasikan, melainkan berfokus pada operasinya dan perilaku yang diharapkan dari tipe data tersebut.

Single LinkedList merupakan struktur data linear yang terdiri dari rangkaian elemen yang disebut node. Setiap node menyimpan dua komponen utama yaitu data (Nilai atau informasi yang disimpan) dan next (pointer/references) alamat memori yang menunjukkan ke node berikutnya. Akses ke seluruh list dimulai dari sebuah pointer khusus yang disebut head.

2.2 Langkah Pengerjaan

- NodeSLL_2511532017
 1. Buka eclipse dan buat package baru dengan nama pekan5_2511532017 dan class NodeSLL_2511532017.
 2. Setelah dibuat, kita akan membuat implementasi dasar Node

```
package pekan5_2511532017;

public class NodeSLL_2511532017 {
    int data_2017;
    NodeSLL_2511532017 next_2017;
    public NodeSLL_2511532017 (int data_2017) {
        this.data_2017 = data_2017;
        this.next_2017 = null;
    }
}
```

3. Disini data_2017 mendefinisikan sebuah kelas dengan tipe int, ini berfungsi untuk menyimpan variabel yang dibawa oleh node

4. NodeSLL_2511532017 next_2017 digunakan mendeklarasikan variabel referensi yang bertipe sama dengan kelasnya sendiri, ini merupakan pointer atau penghubung yang menunjukkan ke node berikutnya.

```
public NodeSLL_2511532017 (int data_2017) {  
    this.data_2017 = data_2017;  
    this.next_2017 = null;  
}
```

5. Ini merupakan sebuah constructor, public NodeSLL_2511532017(int data_2017) akan menerima satu parameter integer yang akan menjadi isi data node, lalu this.data_2017 = data_2017; this disini merujuk pada variabel anggota kelas. this.next_2017 = null; berguna mengatur penghubung ke node berikutnya.

- TambahSLL_2511532017

1. Kita akan membuat class baru untuk TambahSLL_2511532017

```
package pekan5_2511532017;  
  
public class TambahSLL_2511532017 {  
    public static NodeSLL_2511532017 insertAtFront (NodeSLL_2511532017 head_2017, int value_2017) {  
        NodeSLL_2511532017 new_node = new NodeSLL_2511532017 (value_2017);  
        new_node.next_2017 = head_2017;  
        return new_node;  
    }  
}
```

2. InsertAtOn (tambah di awal) disini head_2017 menjadi referensi ke node pertama dan value_2017 akan dimasukkan, lalu buat node baru next node baru ke head lama dan kembalikan node baru sebagai head yang baru.

```
//fungsi menambahkan node di akhir SLL  
public static NodeSLL_2511532017 insertAtEnd (NodeSLL_2511532017 head_2017, int value_2017) {  
    //buat sebuah node dengan sebuah nilai  
    NodeSLL_2511532017 newNode_2017 = new NodeSLL_2511532017 (value_2017);  
    //jika list kosong maka node jadi head  
    if (head_2017 == null) {  
        return newNode_2017;  
    }  
    //simpan head ke variabel sementara  
    NodeSLL_2511532017 last_2017 = head_2017;  
    //telusuri ke node akhir  
    while (last_2017.next_2017 != null) {  
        last_2017 = last_2017.next_2017;  
    }  
    //ubah pointer  
    last_2017.next_2017 = newNode_2017;  
    return head_2017;  
}
```

3. InsertAtEnd jika head == null maka node baru akan langsung jadi head, telusuri list sampai next == null lalu sambungkan node baru. Lalu return head_2017 karena penambahan di akhir tidak mengubah awal list

```
static NodeSLL_2511532017 GetNode_2017 (int data_2017) {
    return new NodeSLL_2511532017 (data_2017);
}
```

4. getNode_2017 disini digunakan untuk membuat dan mengembalikan node baru dengan data tertentu

```
static NodeSLL_2511532017 InsertPos (NodeSLL_2511532017 headNode_2017, int position_2017, int value_2017) {
    NodeSLL_2511532017 head_2017 = headNode_2017;
    if (position_2017 < 1) {
        System.out.println("Invalid position");
    }
    if (position_2017 == 1) {
        NodeSLL_2511532017 new_node = new NodeSLL_2511532017 (value_2017);
        new_node.next_2017 = head_2017;
        return new_node;
    } else {
        while (position_2017-- != 0) {
            if (position_2017 == 1) {
                NodeSLL_2511532017 newNode_2017 = GetNode_2017 (value_2017);
                newNode_2017.next_2017 = headNode_2017.next_2017;
                headNode_2017.next_2017 = newNode_2017;
                break;
            }
            headNode_2017 = headNode_2017.next_2017;
        }
        if (position_2017 != 1) {
            System.out.println("posisi diluar jangkauan");
        }
    }
    return head_2017;
}
```

5. InserPos jika position_2017 < 1 maka print invalid position, lalu jika position_2017 == 1 maka tambahkan node baru pada head dan selama position_2017 != 0 dan jika position_2017 == 1 traversal sampai ke node (posisi - 1) dan sisipkan node baru di antaranya

```
public static void printList (NodeSLL_2511532017 head_2017) {
    NodeSLL_2511532017 curr_2017 = head_2017;
    while (curr_2017.next_2017 != null) {
        System.out.print(curr_2017.data_2017+"-->");
        curr_2017 = curr_2017.next_2017;
    }
    if (curr_2017.next_2017 == null) {
        System.out.println(curr_2017.data_2017);
    } System.out.println();
}
```

6. printList merupakan methode untuk menampilkan isi list dengan traversal dari head dengan diikuti simbol → kecuali node terakhir


```

public static void main (String [] args) {
    //buat linked List 2-->3-->5-->6
    NodeSLL_2511532017 head_2017 = new NodeSLL_2511532017 (2);
    head_2017.next_2017 = new NodeSLL_2511532017 (3);
    head_2017.next_2017.next_2017 = new NodeSLL_2511532017 (5);
    head_2017.next_2017.next_2017.next_2017 = new NodeSLL_2511532017 (6);
    //cetak list asli
    System.out.print("Senarai berantai dari awal : " );
    printList(head_2017);
    //tambahkan node baru didepan
    System.out.print("tambah 1 simpul di depan : ");
    int data_2017 = 1;
    head_2017 = insertAtFront (head_2017, data_2017);
    //cetak update list
    printList (head_2017);
    //tambahkan node baru dibelakang
    System.out.print("tambah 1 simpul ke belakang : ");
    int data2_2017 = 7;
    head_2017 = insertAtEnd (head_2017, data2_2017);
    printList (head_2017);
    System.out.print("tambah 1 simpul ke data 4 : ");
    int data3_2017 = 4;
    int position_2017 = 4;
    head_2017 = insertPos (head_2017, position_2017, data3_2017);
    printList (head_2017);
}
}

```

7. method main untuk membuat list dengan menggunakan head_2017.next_2017 lalu dicetak, lalu tambah nilai di awal dengan menggunakan insertAtFront (head-2017, 1) lalu dicetak, lalu tambah di simpul belakang dengan menggunakan insertAtEnd (head_2017, 7) dan dicetak lalu insertPos untuk menambahkan di simpul data ke 4
8. output

```

Senarai berantai dari awal :2-->3-->5-->6

tambah 1 simpul di depan : 1-->2-->3-->5-->6

tambah 1 simpul ke belakang : 1-->2-->3-->5-->6-->7

tambah 1 simpul ke data 4 : 1-->2-->3-->4-->5-->6-->7

```

- PencarianSLL_2511532017

1. kita akan membuat class baru untuk PencarianSLL_2511532017

```

package pekan5_2511532017;

public class PencarianSLL_2511532017 {
    static boolean searchKey (NodeSLL_2511532017 head_2017, int key_2017) {
        NodeSLL_2511532017 curr_2017 = head_2017;
        while (curr_2017 != null) {
            if (curr_2017.data_2017 == key_2017)
                return true;
            curr_2017 = curr_2017.next_2017;
        }
        return false;
    }
}

```

2. method searchKey static boolean berarti mengembalikan nilai true atau false dengan parameter head_2017 yang merupakan referensi ke node pertama, selama curr_2017 bukan null dan jika data yang dicari sama dengan key maka return true

```
public static void traversal (NodeSLL_2511532017 head_2017) {
    //mulai dari head
    NodeSLL_2511532017 curr_2017 = head_2017;
    //telusuri sampai pointer null
    while (curr_2017 != null) {
        System.out.print(" " + curr_2017.data_2017);
        curr_2017 = curr_2017.next_2017;    }
    System.out.println(); }
}
```

3. method traversal yang berfungsi untuk mencetak ke layar saja dengan tipe return void, curr_2017 merupakan pointer yang menelusur dari awal ke akhir

```
public static void main (String [] args) {
    NodeSLL_2511532017 head_2017 = new NodeSLL_2511532017 (14);
    head_2017.next_2017 = new NodeSLL_2511532017 (21);
    head_2017.next_2017.next_2017 = new NodeSLL_2511532017 (13);
    head_2017.next_2017.next_2017.next_2017 = new NodeSLL_2511532017 (30);
    head_2017.next_2017.next_2017.next_2017.next_2017 = new NodeSLL_2511532017 (10);
    System.out.print("Penelusuran SLL : ");
    traversal (head_2017);
    //data yang akan dicari
    int key_2017 = 30;
    System.out.print("cari data " + key_2017 + " = ");
    if (searchKey(head_2017, key_2017))
        System.out.print("ketemu");
    else
        System.out.print("tidak ada");
    }
}
```

4. method main digunakan untuk membuat list 14, 21, 13, 30, 10 lalu searchkey berupa angka 30
5. Output

```
Penelusuran SLL : 14 21 13 30 10
cari data 30 = ketemu
```

- HapusSLL_2511532017

1. Buat class dengan nama HapusSLL_2511532017

```
package pekan5_2511532017;

public class HapusSLL_2511532017 {
    //fungsi untuk menghapus head
    public static NodeSLL_2511532017 deleteHead_2017 (NodeSLL_2511532017 head_2017) {
        //jika SLL kosong
        if (head_2017 == null)
            return null;
        //pindahkan head ke node berikutnya
        head_2017 = head_2017.next_2017;
        //return head baru
        return head_2017;    }
}
```

2. deleteHead_2017 digunakan untuk menghapus node pertama, jika head_2017 == null maka return null dan pindah ke node berikutnya jika tidak null dan return head_2017

```

public static NodeSLL_2511532017 removeLastNode (NodeSLL_2511532017 head_2017) {
    //jika list kosong, return null
    if (head_2017 == null) {
        return null;
    }
    //jika list satu node, hapus node dan return null
    if (head_2017.next_2017 == null) {
        return null;
    }
    //temukan node terakhir ke dua
    NodeSLL_2511532017 secondLast = head_2017;
    while (secondLast.next_2017.next_2017 != null) {
        secondLast = secondLast.next_2017;
    }
    //hapus node terakhir
    secondLast.next_2017 = null;
    return head_2017;
}

```

3. removeLastNode jika head_2017 == null maka return null , jika cuma ada satu node maka return null , jika tidak maka inisialisasi secondLast lakukan loop jika secondLast.next null maka return head_2017 jika tidak, lanjut iterasi

```

public static NodeSLL_2511532017 deleteNode_2017(NodeSLL_2511532017 head_2017, int pos_2017) {
    NodeSLL_2511532017 temp_2017 = head_2017;
    NodeSLL_2511532017 prev_2017 = null;

    // if linked list is null
    if (temp_2017 == null)
        return head_2017;

    // case 1 : head is deleted
    if (pos_2017 == 1) {
        head_2017 = temp_2017.next_2017;
        return head_2017;
    }

    // kasus 2 : menghapus node di tengah
    // search to the targeted node that will to be deleted
    for (int i_2017 = 1; temp_2017 != null && i_2017 < pos_2017; i_2017++) {
        prev_2017 = temp_2017;
        temp_2017 = temp_2017.next_2017;
    }

    // if founded, delete node
    if (temp_2017 != null) {
        prev_2017.next_2017 = temp_2017.next_2017;
    } else {
        System.out.println("Data tidak ditemukan");
    }

    return head_2017;
}

```

4. deleteNode digunakan untuk menghapus node di posisi tertentu, jika temp_2017 == null maka return head_2017, jika pos_2017 == 1 maka geser head ke node berikutnya, jika remove ditengah maka, loop dimulai dari 1 selama temp_2017 belum null dan belum sampai di posisi target maka prev_2017 maju mengikuti temp_2017 saat ini dan saat sudah di tujuan maka prev_2017.next_2017 = temp_2017.next_2017 dimana prev akan menghubungkan node sebelum node yang dituju dengan node setelah node yang telah dituju.

```

// function to print SLL
public static void printlist_2017(NodeSLL_2511532017 head_2017) {
    NodeSLL_2511532017 curr_2017 = head_2017;
    while (curr_2017.next_2017 != null) {
        System.out.print(curr_2017.data_2017 + "-->");
        curr_2017 = curr_2017.next_2017;
    }
    if (curr_2017.next_2017 == null) {
        System.out.print(curr_2017.data_2017);
    }
    System.out.println();
}

// Driver/Main class

```

5. selama curr_2017 tidak null maka cetak list menggunakan → dan jika null maka cetak tanpa →

```

// Driver/Main class
public static void main(String[] args) {
    // Create SLL 1->2->3->4->5->6->null
    NodeSLL_2511532017 head_2017 = new NodeSLL_2511532017(1);
    head_2017.next_2017 = new NodeSLL_2511532017(2);
    head_2017.next_2017.next_2017 = new NodeSLL_2511532017(3);
    head_2017.next_2017.next_2017.next_2017 = new NodeSLL_2511532017(4);
    head_2017.next_2017.next_2017.next_2017.next_2017 = new NodeSLL_2511532017(5);
    head_2017.next_2017.next_2017.next_2017.next_2017.next_2017 = new NodeSLL_2511532017(6);

    // print early list
    System.out.print("list awal : ");
    printlist_2017(head_2017);

    // delete head
    head_2017 = deleteHead_2017(head_2017);
    System.out.print("list setelah simpul head di hapus : ");
    printlist_2017(head_2017);

    //hapus node terakhir
    head_2017 = removeLastNode(head_2017);
    System.out.print("list setelah simpul terakhir di hapus : ");
    printlist_2017(head_2017);

    // delete node at pos [2]
    int pos2_2017 = 2;
    head_2017 = deleteNode_2017(head_2017, pos2_2017);
    // print list after deletion
    System.out.print("list setelah posisi 2 dihapus : ");
    printlist_2017(head_2017);
}

```

6. Method main buat list 1,2,3,4,5,6 sebagai list awal lalu kita hapus head dengan menggunakan deleteHead lalu cetak, lalu removeLastNode untuk menghapus node terakhir dan deleteNode pos2_2017 = 2 (posisi 2)
7. Output

```

list awal : 1-->2-->3-->4-->5-->6
List setelah simpul head di hapus : 2-->3-->4-->5-->6
List setelah simpul terakhir di hapus : 2-->3-->4-->5
List setelah posisi 2 dihapus : 2-->4-->5

```

Latihan

- Pasien_2511532017
 - Kita akan membuat class baru untuk Pasien_2511532017

```
package pekan5_2511532017;

public class Pasien_2511532017 {
    private String namaPasien_2017;
    private String penyakit_2017;
    private int antrian_2017;
    private Pasien_2511532017 next_2017;
```

- Menyimpan nama, penyakit, nomor antrian dan pointer dengan menggunakan private

```
//constructor
public Pasien_2511532017 (String namaPasien_2017, String penyakit_2017, int antrian_2017) {
    this.namaPasien_2017 = namaPasien_2017;
    this.penakit_2017 = penyakit_2017;
    this.antrian_2017 = antrian_2017;
}
```

- Constructor dipanggil saat membuat objek node baru, this membedakan antara atribut class dengan parameter method

```
//setter getter
public String getNamaPasien_2017 () {return namaPasien_2017;}
public void setNamaPasien_2017 (String namaPasien_2017) {this.namaPasien_2017= namaPasien_2017;}

public String getPenyakit_2017 () { return penyakit_2017;}
public void setPenyakit_2017 (String penyakit_2017) {this.penakit_2017 = penyakit_2017;}

public int getAntrian_2017 () { return antrian_2017;}
public void setAntrian_2017 (int antrian_2017) {this.antrian_2017 = antrian_2017;}

public Pasien_2511532017 getNext_2017 () {return next_2017;}
public void setNext_2017 (Pasien_2511532017 next_2017) {this.next_2017 = next_2017;}
```

- Setter untuk membaca nilai atribut private dan getter mengubah nilai atribut private

```
//tambah pasien
public static Pasien_2511532017 insertAtEnd_2511532017 (Pasien_2511532017 head_2017, String namaPasien_2017, String penyakit_2017, int antrian_2017) {
    //buat sebuah node dengan sebuah nilai
    Pasien_2511532017 newNode_2017 = new Pasien_2511532017 (namaPasien_2017, penyakit_2017, antrian_2017);
    //jika list kosong maka node jadi head
    if (head_2017 == null) {
        return newNode_2017;
    }
    //simpan head ke variabel sementara
    Pasien_2511532017 last_2017 = head_2017;
    //telusuri ke node akhir
    while (last_2017.next_2017 != null) {
        last_2017 = last_2017.next_2017;
    }
    //ubah pointer
    last_2017.next_2017 = newNode_2017;
    return head_2017;
}
```

- Membuat node baru dengan data yang diterima dari inputan user, jika list kosong maka node baru akan langsung jadi head, jika list berisi maka telusuri node hingga next nya null dan hubungkan node terakhir dengan node baru melalui pointer next_2017

```
//delete head (panggil pasien dari antrian)
public static Pasien_2511532017 deleteHead_2511532017 (Pasien_2511532017 head_2017) {
    //jika SLL kosong
    if (head_2017 == null)
        return null;
    System.out.println("pasien berhasil dipanggil");
    System.out.println(head_2017.toString());
    //pindahkan head ke node berikutnya
    head_2017 = head_2017.next_2017;
    //return head baru
    return head_2017;
}
```

6. Method panggil pasien, cek jika list kosong, jika ya return tampilkan informasi dengan toString dan pindahkan reference ke node berikutnya dan return head baru

```
//tampilkan antrian
public static void printList_2511532017 (Pasien_2511532017 head_2017) {
    Pasien_2511532017 curr_2017 = head_2017;
    while (curr_2017.next_2017 != null) {
        System.out.print(curr_2017.namaPasien_2017+" --> ");
        curr_2017 = curr_2017.next_2017;
    }
    if (curr_2017.next_2017 == null) {
        System.out.println(curr_2017.namaPasien_2017);
    } System.out.println();
}
```

7. Mehtod untuk menampilkan dengan traversal dari awal menggunakan pointer, loop selama next_2017 tidak null maka pakai → jika next_2017 null maka tidak pakai →

```
//cari nama pasien
static boolean searchKey_2511532017 (Pasien_2511532017 head_2017, String key_2017) {
    Pasien_2511532017 curr_2017 = head_2017;
    while (curr_2017 != null) {
        if (curr_2017.namaPasien_2017.equalsIgnoreCase(key_2017)) {
            System.out.println("pasien ditemukan");
            System.out.println(curr_2017.toString());
            return true;
        }
        curr_2017 = curr_2017.next_2017;
    }
    System.out.println("pasien dengan nama "+ key_2017 + "tidak ditemukan");
    return false;
}

public static void traversal (Pasien_2511532017 head_2017) {
    //mulai dari head
    Pasien_2511532017 curr_2017 = head_2017;

    if (curr_2017 == null) {
        System.out.println("list kosong");
        return;
    }
    //telusuri sampai pointer null
    while (curr_2017 != null) {
        System.out.print(" " + curr_2017.namaPasien_2017);
        curr_2017 = curr_2017.next_2017;
    }
    System.out.println();
}
```

8. Method cari pasien, traversal seluruh list dan bandinngkan dengan key menggunakan equalsIgnoreCase() untuk pencarian case insensitive jika ditemukan print pesan sukses dan detail pasien dan jika tidak ditemukan maka print pesan tidak ditemukan. Method traversal, cek jika kosong, tampilkan psan dan keluar

```
//menampilkan jumlah antrian
public static void jumlahPasien_2511532017 (Pasien_2511532017 head_2017) {
    int count_2017 = 0;
    Pasien_2511532017 curr_2017 = head_2017;

    while (curr_2017 != null) {
        count_2017++;
        curr_2017 = curr_2017.next_2017;
    }

    if (count_2017 == 0) {
        System.out.println("antrian kosong");
    } else {
        System.out.println("total pasien dalam antrian: " + count_2017);
    }
    // Tampilkan status lengkap
    System.out.println("STATUS ANTRIAN:");
    System.out.println("Total pasien: " + count_2017);
    System.out.println("Pasien terdepan: " + head_2017);
}

```

9. Method jumlah pasien dan informasi antrian pertama, loop melalui seluruh list, increment counter untuk setiap node. Tampilkan ringkasan status antrian termasuk informasi pasien terdepan.

```
@Override
public String toString() {
    return "\n Nama Pasien: " + namaPasien_2017 +
        "\nPenyakit/keluhan: " + penyakit_2017 +
        "\nAntrian: " + antrian_2017;
}

```

10. Method toString yang bawaan Java

- RumahSakit_2511532017
 1. Kita buat kelas untuk RumahSakit_2511532017
 2. Method tampilkan menu memberikan 6 pilihan menu
 3. Inisialisai variabel, dan struktur loop do while yang menjamin menu akan tampil minimal satu kali sebelum pengecekan kondisi
 4. Method menu handling sc.nextLine() membaca seluruh baris input sebagai string, trim() menghapus spasi dan Integer.parseInt() mengkonversi String menjadi Integer. Jika input bukan angka valid maka NumberFormatException
 5. Case 1 meminta input nama pasien, penyakit dan antrian dan panggil method insertAtEnd dan update head_2017
 6. Case 2 memanggil deleteHead_2511532017 untuk menghapus node terdepan dan head diupdate
 7. Case 3 printList menampilkan semua list

8. Case 4 meminta input untuk mencari nama pasien dengan memanggil method `searchKey_2511532017` dengan parameter `head` dan `keyword` pencarian
9. Case 5 cek status antrian dengan memanggil method `jumlahPasien_2511532017` untuk menampilkan statis antrian dan informasi posisi terdepan
10. Case 6 keluar dari menu
11. Output

Input nama

```
=== Program Antrian Rumah Sakit NIM: 2511532017 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu (1-6): 1
Masukkan nama pasien: mondo
Masukkan penyakit/keluhan: demam
Masukkan nomor antrian: 1

=== Program Antrian Rumah Sakit NIM: 2511532017 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu (1-6): 1
Masukkan nama pasien: dudu
Masukkan penyakit/keluhan: batuk
Masukkan nomor antrian: 2

=== Program Antrian Rumah Sakit NIM: 2511532017 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu (1-6): 1
Masukkan nama pasien: dono
Masukkan penyakit/keluhan: flu
Masukkan nomor antrian: 3
```

Tampilkan antrian


```
=== Program Antrian Rumah Sakit NIM: 2511532017 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu (1-6): 3
mondo --> dudu --> dono
```

Cari pasien

```
=== Program Antrian Rumah Sakit NIM: 2511532017 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu (1-6): 4
Masukkan nama pasien yang dicari: mondo
pasien ditemukan

Nama Pasien: mondo
Penyakit/keluhan: demam
Antrian: 1
```

Cek status antrian

```
=== Program Antrian Rumah Sakit NIM: 2511532017 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu (1-6): 5
total pasien dalam antrian: 3
STATUS ANTRIAN:
Total pasien: 3
Pasien terdepan:
Nama Pasien: mondo
Penyakit/keluhan: demam
Antrian: 1
```

Pangiil pasien

```
=== Program Antrian Rumah Sakit NIM: 2511532017 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilih menu (1-6): 2
pasien berhasil dipanggil

Nama Pasien: mondo
Penyakit/keluhan: demam
Antrian: 1
```

BAB III

KESIMPULAN

Struktur data Single LinkedList berhasil diimplementasikan menggunakan prinsip LIFO (Last In First Out). Semua operasi fundamental seperti insertAtEnd, delete head dan searching telah dilaksanakan dengan baik. Dengan percobaan ini saya berhasil memahami setiap node terdiri dari data dan pointer ke node berikutnya.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Linked list data structure," GeeksforGeeks.
<https://www.geeksforgeeks.org/linked-list-data-structure/>
(accessed May 05, 2026).